Graphical User Interface and PC–ESP32 Communication Standard for the SOFIA Vision System

Marco Heimdal Castro Magaña, Alejandro Amador Franco, Karen Alejandra Guevara Vasquez, María Alicia Vela Contreras, Juan Pablo Contreras Quiñones, Jorge A. Lizarraga, Luis E. González-Jiménez, Luis F. Luque-Vega

***Abstract*—The SOFIA modular kinetic surface uses an external computer vision system that recognizes five human gestures to drive its actuators. This work presents the graphical user interface (GUI) that displays the live video and the detection result, and the wireless communication standard that transmits the recognized gestures from the host PC to the ESP32 master controller. The GUI was implemented with PySide6 and separates the interface, the video processing, and the Bluetooth Low Energy (BLE) link into independent threads, so that the detection can be started, paused, resumed, and stopped without blocking user interaction. The communication standard defines a frame of 5 to 19 bytes with a sequence number and a CRC-8 checksum, five commands, positive and negative acknowledgements, six error codes, and rules for retransmission, duplicate rejection, and link supervision. An ESP32 firmware was developed to receive, validate, and acknowledge the frames, and to map each gesture to a distinct motion pattern on one two-degree-of-freedom SOFIA unit. The implementation was verified through software tests and a simulation of the firmware; validation on the physical hardware remains as the next step.**

***Index Terms*—Graphical user interface, multithreading, Bluetooth Low Energy, communication protocol, ESP32, gesture recognition, modular robotics.**

I. Introduction

SOFIA is a modular robotic installation composed of 16 mechatronic units arranged in a honeycomb configuration, where each unit provides two rotational degrees of freedom actuated by servomotors [1]. Interaction with the user is achieved through an external computer vision system that recognizes five gestures (INICIO, RECARGAR, ESCUDO, ATAQUE, and FIN) using MediaPipe Pose Landmarker and Hand Landmarker [2], and that also tracks the position of the user's hand.

Before this work, the vision system operated as an independent prototype: the result was shown in an OpenCV window controlled through the keyboard, and the recognized gestures were not transmitted to the embedded controller. Integrating the vision system with the robot requires two elements. First, a GUI that allows an operator to visualize the detection and control the processing without freezing the interface, since landmark detection takes tens of milliseconds per frame. Second, a communication standard that delivers the recognized gestures to the ESP32 master controller [3] reliably over a wireless link.

This work addresses four activities: (i) the design of the GUI architecture and wireframes, including the justification of the framework; (ii) the integration of the detection result into the GUI using threads; (iii) the implementation of controls to start, stop, and resume the detection; and (iv) the design and implementation of the PC–ESP32 communication standard. The remainder of this paper is organized as follows. Section II presents the GUI design, Section III describes its implementation, Section IV defines the communication standard, Section V presents the ESP32 firmware, Section VI reports the verification, and Section VII concludes the work.

The authors are affiliated with the Instituto Tecnológico de Estudios Superiores de Occidente (ITESO), Tlaquepaque, Jalisco, Mexico.

TABLE I
Comparison of GUI Frameworks

Criterion	**PySide6 (Qt)**	Tkinter / CustomTkinter	Web (Flask + browser)
Threads integrated with the GUI	Native (`QThread`, thread-safe signals)	Manual (queues and polling)	Separate server process
Video display	Direct (`QImage`)	Through PIL conversion	MJPEG streaming
Synchronization primitives	`QMutex`, `QWaitCondition`	Not included	Not included
Styling	Style sheets (QSS)	Basic / moderate	Extensive
License	LGPL	PSF / MIT	–

II. GUI Architecture and Design

A. Framework Selection

The main requirement of the GUI is to display video at the camera frame rate while the landmark detection runs, without blocking the interaction with the user. The alternatives considered are compared in Table I.

PySide6, the official Python binding of Qt [4], was selected because it provides the mechanisms required by the problem: `QThread` to move the processing out of the interface thread, signals and slots to transfer data between threads safely, and `QMutex` with `QWaitCondition` to pause a thread without busy waiting. In addition, frames captured by OpenCV can be converted directly into a `QImage` without intermediate libraries, and its LGPL license is compatible with an academic project.

B. Software Architecture

The application is organized in modules with a single responsibility each, as shown in Fig. 1 and Table II. The detection logic is encapsulated in a pipeline shared by the GUI and by the original keyboard-based application, so that both use exactly the same implementation.

C. Threading Model

The application uses three threads (Table III). The GUI thread only draws widgets and handles user events. The vision

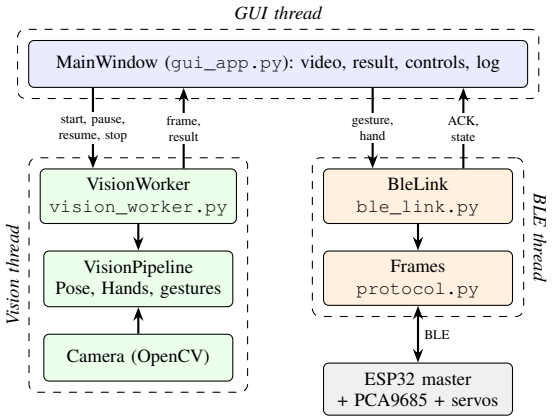


Fig. 1. Software architecture of the GUI. Each dashed region runs in its own thread.

TABLE II
MAIN SOFTWARE COMPONENTS

Module	Responsibility
`gui_app.py`	Window, controls, and visualization; it does not process video
`vision_worker.py`	Thread that reads the camera and runs the pipeline; implements pause, resume, and stop
`vision_pipeline.py`	Processes one frame according to the active mode (game or hand tracking)
`gesture_recognition.py`	Decides the recognized gesture from the landmarks
`hand_tracker.py`	Hand position in a Cartesian plane centered on the image
`ble_link.py`	BLE connection, transmission with acknowledgement, and retransmission
`protocol.py`	Construction and validation of frames

TABLE III
THREADS OF THE APPLICATION

Thread	Task	Communication with the GUI
GUI	Draws the window and handles clicks	–
Vision	Camera capture and landmark detection	Qt signals (queued connections); commands protected by `QMutex`
BLE	Scanning, connection, transmission, and acknowledgement wait	Coroutines scheduled in the event loop from the GUI thread; results through Qt signals

thread captures frames and runs MediaPipe, which requires approximately 20 to 40 ms per frame. The BLE thread runs an `asyncio` event loop for the Bleak library [5], since scanning, connecting, and waiting for acknowledgements may take from milliseconds to seconds.

Communication from the vision thread to the GUI uses Qt signals, which Qt delivers in the receiving thread through queued connections. In the opposite direction, the GUI calls methods of the worker that modify state variables protected by a `QMutex`. In this way, no widget is accessed outside the GUI thread and no shared variable is accessed without synchronization.

D. Wireframe and Layout

The main window is shown in Fig. 2. The video occupies the left side, since it is the main element of the interface, while a fixed-width column on the right groups the controls in the order in which they are used: detection control, mode, result, ESP32 connection, and event log.

TABLE IV
DESIGN DECISIONS OF THE GUI

Decision	Justification
Large video, fixed right column	The video is the main element; the controls remain visible
Grouped panels	Each group corresponds to one task, ordered by the usage flow
Buttons enabled by state	Prevents invalid actions, such as resuming a stopped process
Color state indicators	The state of the thread and of the link is recognized at a glance
Large result label	The gesture can be read at a distance from the camera
Event log	Shows every frame sent and its acknowledgement; useful as evidence and for debugging
“Send results” option	Allows testing the vision system without moving the robot
Shared detection pipeline	The GUI runs the same code that was evaluated in the keyboard application

Algorithm 1 Processing loop of the vision thread

Require: Camera index, initial mode
Open camera and load Pose and Hand Landmarker
Emit state RUNNING
loop
  Lock mutex
  if *paused* **and** *running* **then**
    Emit state PAUSED
    while *paused* **and** *running* **do**
      Wait on condition (no CPU use)
    end while
  end if
  $m \leftarrow$ requested mode; unlock mutex
  if not *running* **then break**
  end if
  Capture frame; process it with the pipeline in mode m
  Emit frame, result, FPS, and latency to the GUI
end loop
Release camera and models; emit state STOPPED

The interaction flow is as follows: the operator optionally connects to the ESP32 and sends a PING to verify the link, selects the mode, and starts the detection. During the detection, the operator can pause, resume, or stop the processing, and change the mode, which is applied from the next frame. The design decisions are summarized in Table IV.

III. DETECTION VISUALIZATION AND CONTROLS

A. Processing Thread

The processing is performed by `VisionWorker`, a subclass of `QThread`. The camera and the MediaPipe models are created inside the thread that uses them, and each processed frame is sent to the GUI as a `QImage` together with the detection result and the frame rate. The loop of the thread is summarized in Algorithm 1.

B. Detection Controls

Each control produces a specific effect on the processing thread, as described in Table V, and the resulting state

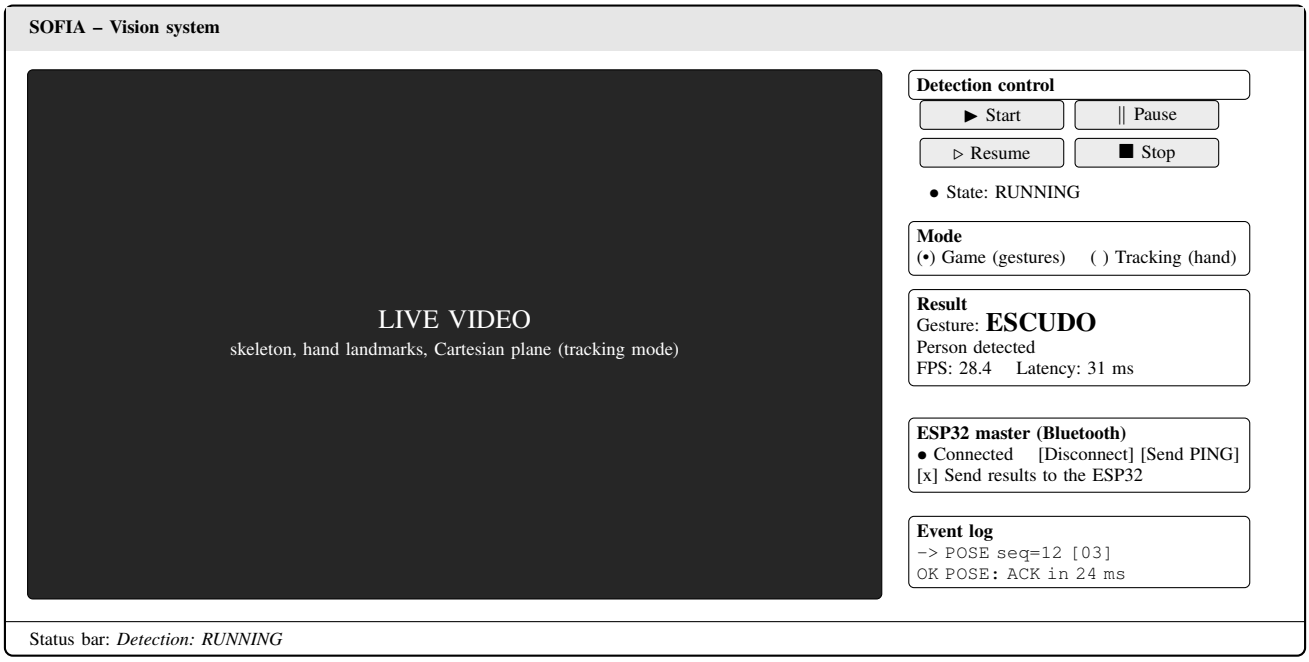


Fig. 2. Wireframe of the main window. The right column keeps every control visible without covering the video.

TABLE V
EFFECT OF EACH CONTROL ON THE PROCESSING THREAD

Control	Effect on the thread	Camera
Start	Creates the worker and calls `start()`; the camera and models are loaded inside the thread	Opened
Pause	Sets *paused*; the thread blocks on `QWaitCondition` at the next iteration	Open (video frozen)
Resume	Clears *paused* and calls `wakeAll()`; the loop continues	Open
Stop	Clears *running* and calls `wakeAll()`; the loop ends and resources are released; a STOP frame is sent to the ESP32	Closed

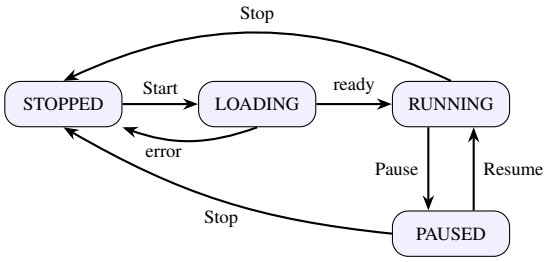


Fig. 3. States of the processing thread and the controls that produce each transition.

transitions are shown in Fig. 3. The pause is implemented with a `QWaitCondition`: the thread is blocked without consuming CPU until `resume()` or `stop()` wakes it up. The buttons are enabled according to the current state, so only valid transitions can be requested.

C. Visualization of the Result

In game mode, the result panel shows the recognized gesture and whether a person is detected. In tracking mode, Pose detection and gesture recognition are disabled; the panel shows the position of the hand in a Cartesian plane whose origin is the center of the image, normalized to $[-1, 1]$ on each axis, together with the magnitude and angle of the vector from the origin to the hand. The video shows the landmarks and, in tracking mode, the plane and the vector.

TABLE VI
BLE LINK PARAMETERS

Parameter	Value
Advertised name	`SOFIA-ESP32`
Service	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`
RX (PC → ESP32)	`6E400002-...`, write without response
TX (ESP32 → PC)	`6E400003-...`, notify
Maximum frame	19 bytes (one BLE write)

IV. COMMUNICATION STANDARD

A. Link Layer

The ESP32-C3 supports Bluetooth Low Energy but not Bluetooth Classic [3]; therefore, the link uses BLE [6] with the Nordic UART Service (NUS) profile [7], a de facto standard that can also be tested with generic BLE applications. The ESP32 acts as a peripheral (GATT server) and the PC as a central (GATT client). The parameters are listed in Table VI. Each BLE write carries exactly one complete frame, and the longest frame (19 bytes) fits in the default BLE payload of 20 bytes, so no MTU negotiation is required.

B. Frame Structure

The frame structure is shown in Fig. 4 and its fields are described in Table VII. Multi-byte integers are transmitted in little-endian order.

The checksum is a CRC-8 with polynomial `0x07` ($x^8 + x^2 + x + 1$), initial value `0x00`, no reflection, and no final

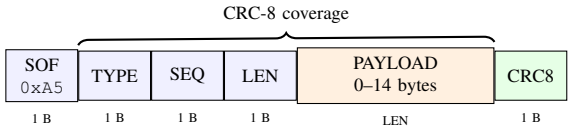


Fig. 4. Frame structure of the communication standard.

TABLE VII
FRAME FIELDS

Field	Size	Description
SOF	1 B	Start of frame, always `0xA5`
TYPE	1 B	Message type (Table VIII)
SEQ	1 B	Sequence number 0–255; each side keeps its own counter
LEN	1 B	Payload length, 0–14
PAYLOAD	0–14 B	Message data
CRC8	1 B	CRC-8 over TYPE, SEQ, LEN, and PAYLOAD

TABLE VIII
MESSAGE TYPES

TYPE	Name	Direction	Payload	ACK
`0x00`	POSE	PC → ESP32	1 B	Yes
`0x01`	TRACK	PC → ESP32	9 B	No
`0x02`	PING	PC → ESP32	0 B	Yes
`0x03`	MODE	PC → ESP32	1 B	Yes
`0x04`	STOP	PC → ESP32	0 B	Yes
`0x80`	ACK	ESP32 → PC	2 B	–
`0x81`	NACK	ESP32 → PC	3 B	–

TABLE IX
TRACK PAYLOAD

Bytes	Field	Type	Encoding and range
0	hand	u8	0 left, 1 right
1–2	x	i16	$x_{\text{norm}} \times 1000$, -1000 to 1000
3–4	y	i16	$y_{\text{norm}} \times 1000$, -1000 to 1000
5–6	$\lvert v\rvert$	u16	$\lvert v\rvert_{\text{norm}} \times 1000$, 0 to 1415
7–8	angle	u16	degrees $\times 10$, 0 to 3599

XOR. With these parameters, CRC8("123456789") = `0xF4`, which is used as a check value for any implementation.

C. Commands and Payloads

The message types are listed in Table VIII. The POSE command carries the identifier of the recognized gesture (1 INICIO, 2 RECARGAR, 3 ESCUDO, 4 ATAQUE, 5 FIN) and is sent only when a new gesture appears. The TRACK command carries the hand position (Table IX) normalized with respect to the image, so it does not depend on the camera resolution. Since TRACK is a continuous stream in which the most recent value supersedes the previous ones, it does not require acknowledgement.

The PING command has no payload and is used both as the test command and as a heartbeat. The MODE command informs the active mode (0 game, 1 test, 2 tracking), and the STOP command requests the ESP32 to enter a safe state.

D. Acknowledgements and Error Codes

The ESP32 answers each valid command that requires confirmation with an ACK whose payload contains the type and sequence number of the confirmed command. Invalid frames are answered with a NACK that contains the received type, the received sequence number, and an error code (Table X). The ESP32 validates each frame in the order length, SOF, LEN, CRC, type, and payload, and reports the first error found.

TABLE X
ERROR CODES

Code	Name	Cause	PC retries
`0x01`	`ERR_SOF`	First byte is not `0xA5`	No
`0x02`	`ERR_LENGTH`	Frame size or LEN invalid	No
`0x03`	`ERR_CRC`	Checksum mismatch	Yes
`0x04`	`ERR_UNKNOWN_TYPE`	Undefined TYPE	No
`0x05`	`ERR_PAYLOAD`	Payload size or values out of range	No
`0x06`	`ERR_BUSY`	Reception queue full	Yes

TABLE XI
COMMUNICATION RULES

Rule	Description
R1	Commands with ACK are sent one at a time (stop-and-wait)
R2	ACK timeout of 300 ms
R3	Up to three transmissions with the same SEQ; then the command is reported as failed
R4	A command with the same TYPE and SEQ as the last executed one, received within 1 s, is acknowledged again but not executed
R5	An ACK/NACK is valid only if its TYPE and SEQ match the pending command
R6	TRACK is limited to 20 frames per second
R7	If the PC has not transmitted for 1 s, it sends a PING (heartbeat)
R8	If no valid frame is received for 3 s, or the link is lost, the ESP32 enters a safe state
R9	On connection the PC sends MODE; when the detection stops the PC sends STOP

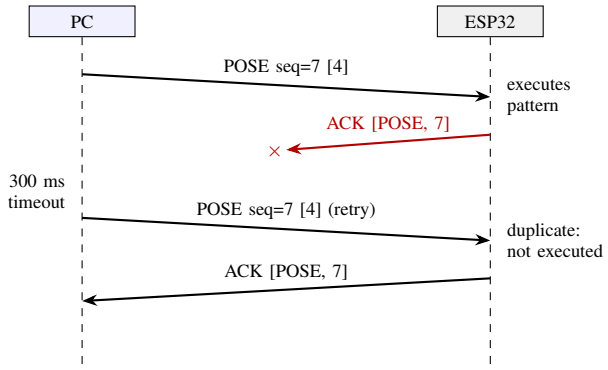


Fig. 5. Retransmission after a lost acknowledgement (rules R2–R4).

E. Communication Rules

The rules of Table XI make the communication robust against lost frames, corrupted frames, and loss of the link. Fig. 5 illustrates the case in which an acknowledgement is lost: the PC retransmits the command with the same sequence number, and the ESP32 recognizes it as a duplicate, so the unit does not execute the same motion twice.

F. Example Frames

Table XII lists frames generated by the PC implementation. For instance, the TRACK frame encodes the right hand at $x = 0.25$, $y = -0.40$, $|v| = 0.472$, and an angle of $302.0°$.

TABLE XII
EXAMPLE FRAMES (HEXADECIMAL)

Message	SEQ	Frame
PING	1	`A5 02 01 00 C3`
POSE ESCUDO	2	`A5 00 02 01 03 CA`
TRACK	3	`A5 01 03 09 01 FA 00 70 FE D8 01 CC 0B 1E`
MODE (track.)	4	`A5 03 04 01 02 8A`
STOP	5	`A5 04 05 00 EA`
ACK of PING 1	0	`A5 80 00 02 02 01 6C`
NACK (CRC)	1	`A5 81 01 03 02 04 03 16`

Algorithm 2 Frame processing in the ESP32

Require: Received frame f of n bytes
if $n < 5$ **or** $n > 19$ **then**
  return NACK(`ERR_LENGTH`)
end if
if $f_0 \neq$ `0xA5` **then**
  return NACK(`ERR_SOF`)
end if
if $f_3 > 14$ **or** $n \neq f_3 + 5$ **then**
  return NACK(`ERR_LENGTH`)
end if
if CRC8$(f_1 \dots f_{n-2}) \neq f_{n-1}$ **then**
  return NACK(`ERR_CRC`)
end if
Update link watchdog
if TYPE unknown **then**
  return NACK(`ERR_UNKNOWN_TYPE`)
end if
if payload invalid **then**
  return NACK(`ERR_PAYLOAD`)
end if
if not duplicate(TYPE, SEQ) **then**
  Execute command; store (TYPE, SEQ)
end if
if TYPE $\neq$ TRACK **then**
  return ACK(TYPE, SEQ)
end if

V. ESP32 MASTER FIRMWARE

A. Reception and Validation

The firmware was developed in the Arduino environment for the ESP32-C3. It advertises the NUS service, and the BLE write callback only copies each received frame into a FreeRTOS queue; the validation and the execution are performed in the main loop, which keeps the BLE task short. Algorithm 2 summarizes the processing of each frame.

B. Motion Patterns

To demonstrate the end-to-end operation, each gesture was mapped to a distinct motion of one SOFIA unit with two servomotors driven through a PCA9685 [8], [9], using the same pin assignment and angle-to-PWM calibration as the servo characterization tests ($-60°$ to $+60°$). Servo channel 0 moves the unit horizontally and channel 1 vertically. The patterns are listed in Table XIII.

TABLE XIII
MOTION ASSOCIATED WITH EACH COMMAND

Command	Motion	Detail
POSE 1 INICIO	Circle	2 turns, radius $35°$
POSE 2 RECARGAR	Square	2 turns, side $70°$
POSE 3 ESCUDO	X shape	Both diagonals through the center
POSE 4 ATAQUE	Vertical line	3 strokes of $\pm 35°$
POSE 5 FIN	Zigzag	Left to right
TRACK (tracking)	Hand following	$x, y \times 45°$, up to $300°$/s
STOP	Return to center	Interrupts any pattern

TABLE XIV
VERIFICATION RESULTS

Test	Observed result	Status
Controls of the thread	States: loading, running, paused, running, stopped; no frames emitted while paused	**Passed**
CRC-8 (PC and ESP32)	Check value `0xF4` and identical CRC for all example frames	**Passed**
Command with ACK	PING and POSE acknowledged	**Passed**
Lost ACK (R3, R4)	Retransmitted with the same SEQ; executed once	**Passed**
No response (R3)	Three attempts, then reported as failed	**Passed**
Invalid payload	NACK `ERR_PAYLOAD`, no retransmission	**Passed**
TRACK rate (R6)	10 frames in 0.5 s (20 Hz)	**Passed**
Heartbeat (R7)	PING sent after 1 s without traffic	**Passed**
Motion patterns	Five distinct trajectories, all ending at the center (Fig. 6)	**Passed**
Physical hardware	BLE link and servo motion on the ESP32-C3	**Pending**

Each pattern is defined as a list of waypoints in normalized coordinates that is scaled to $\pm 35°$. Every 20 ms the controller advances the commanded position toward the next waypoint at a constant speed of $120°$/s, converts each angle to PWM counts, and updates the PCA9685 only when the change exceeds $0.1°$. Every pattern ends at the center, and a new gesture interrupts the current pattern. In the safe state, a pattern in progress is allowed to finish, since it ends at the center, while any other motion returns immediately to the center.

VI. VERIFICATION

The implementation was verified at the software level; the tests are summarized in Table XIV. The GUI was executed with the camera and the detection pipeline replaced by stubs, and the controls were applied in sequence. The BLE link was tested against a simulated ESP32 that reproduces the validation and duplicate-rejection rules of the firmware. The CRC-8 of the firmware was compiled and compared with the PC implementation, and the complete firmware was compiled on a PC with simulated Arduino, BLE, and PCA9685 interfaces, injecting the same frames that the PC sends and recording the PWM commands of each servo.

Fig. 6 shows the trajectories obtained from the simulated firmware, computed by converting the PWM counts sent to each servo back into angles. The five gestures produce distinguishable trajectories, and the tracking mode follows the commanded hand position. The staircase appearance corresponds to the resolution of the PCA9685 with the adopted calibration, in which 160 counts cover $120°$ ($\approx 0.75°$ per count). The simulated durations of the patterns range from 2.4 s (FIN) to 5.6 s (RECARGAR).

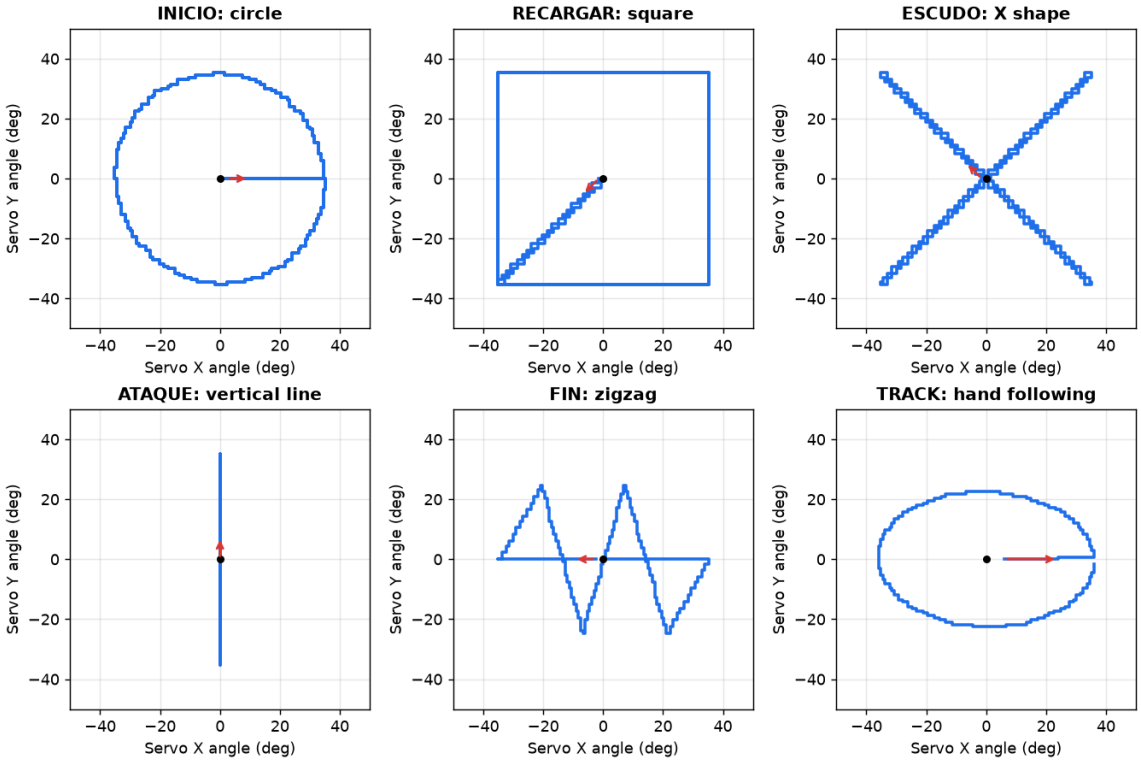


Fig. 6. Trajectories of the unit generated by the simulated ESP32 firmware for each command. The black dot marks the center and the red arrow the initial direction.

The procedure to validate the system on the physical hardware consists of loading the firmware on the ESP32-C3, running the test script on the PC, which sends a PING, a POSE, a TRACK, and a frame with a corrupted CRC, and verifying that the PC receives an ACK, an ACK, no response, and a NACK with `ERR_CRC`, respectively, while the unit draws the X pattern associated with ESCUDO.

VII. Conclusions

A GUI was developed for the SOFIA vision system using PySide6. Separating the interface, the video processing, and the BLE communication into three threads keeps the interface responsive, and the start, pause, resume, and stop controls act directly on the processing thread through a mutex and a wait condition. A communication standard was defined to transmit the recognized gestures and the hand position from the PC to the ESP32 master; its frame structure, CRC-8 checksum, acknowledgements, error codes, and retransmission and supervision rules address lost frames, corrupted frames, and loss of the link. An ESP32 firmware that receives, validates, and acknowledges the frames was implemented and associated with five distinct motion patterns of a two-degree-of-freedom unit. The software verification confirmed the expected behavior of the controls, the protocol rules, and the motion patterns. Future work includes the validation on the physical hardware, the measurement of the end-to-end latency from gesture to motion, and the extension of the motion commands to the 16 modules of SOFIA.

References

[1] J. A. Lizárraga, “Presentación del labdst: Sesión 1 - contexto, metodología d-lab y sofia,” 2026, iTESO, 17 de agosto de 2026.
[2] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, “MediaPipe: A framework for building perception pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
[3] Espressif Systems, “ESP32-C3 Series Datasheet, Version 2.4,” May 2026, accessed: 2026-08. [Online]. Available: https://documentation.espressif.com/ESP32-C3_Datasheet_en.pdf
[4] The Qt Company, “Qt for Python (PySide6) documentation,” 2026. [Online]. Available: https://doc.qt.io/qtforpython-6/
[5] H. Blidh, “Bleak: Bluetooth low energy platform agnostic klient,” 2026. [Online]. Available: https://github.com/hbldh/bleak
[6] Bluetooth SIG, “Bluetooth Core Specification, version 5.4,” 2023.
[7] Nordic Semiconductor, “Nordic UART Service (NUS),” 2026.
[8] NXP Semiconductors, “PCA9685 - 16-channel, 12-bit PWM Fm+ I2C-bus LED controller, Product data sheet Rev. 4,” Apr. 2015, 16 April 2015. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf
[9] Adafruit Industries, “Adafruit PWM Servo Driver Library,” 2026. [Online]. Available: https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library